Below are the prompts and corresponding responses generated using ChatGPT for the development of the BERT prediction aggregation algorithm.

Prompt:
Help mw write the below algoritam in Python to read filght phases from a .txt file, And determine the total fight phases.
The input from the txt file is like shown below
[29:23 → 29:50] : [00:27] : Preflight (Average Confidence: 0.948)
[29:50 → 30:44] : [00:54] : Climb (Average Confidence: 0.495)
[30:44 → 45:22] : [14:38] : CONVERSATION
[45:22 → 46:30] : [01:08] : Preflight (Average Confidence: 0.237)
[46:30 → 51:57] : [05:27] : CONVERSATION
[51:57 → 52:36] : [00:39] : TAXI (Average Confidence: 0.837)
[52:36 → 59:18] : [06:42] : CONVERSATION
[59:18 → 59:59] : [00:41] : TAXI (Average Confidence: 0.480)
[01:00:00 → 01:06:26] : [06:26] : CONVERSATION
[01:06:31 → 01:07:11] : [00:40] : Takeoff (Average Confidence: 0.485)
[01:07:11 → 01:07:54] : [00:43] : CONVERSATION
[01:07:54 → 01:08:47] : [00:53] : Preflight (Average Confidence: 0.494)

Algorithum
Phase_order[] = Preflight, TAXI, Takeoff, Climb, Cruise, Descent, Approach, Landing, TAXI, Parking;
Min_phase_time_limit[] = 05:00, 01:30, 00:45, 10:00, 20:00, 05:00, 03:00, 00:30, 02:00, 02:00;
Max_phase_time_limit[] = 45:00, 30:50, 03:45, 40:00, 02:20:00, 40:00, 30:00, 03:30, 20:00, 20:00;
Inital_phase = NULL

loop Start

Once you get the initial phase continue the next steps,

```
If(Inital_phase != NULL)
        Total_Duration = Current_Phase_duration + Duration

        if(Phase == Current_phase)      //Adding both the phase.
                Current_phase_end_time = End_time;
                if(Duration > 01:00)
                        Current_phase_confidence = (Confidence + Current_phase_confidence)/2
        else
                if(Phase == CONVERSATION)
                        if(duration < 05:00)
                                if(Total_Duration < Max_phase_time_limit[current_phase])
                                        Current_phase_end_time = End_time;
                                        Current_Phase_duration = Total_Duration;
                                else
                                        Phases[] = Current_phase;
                                        Durations[] = Current_Phase_duration
                                        Confidences[] = Current_phase_confidence
                                        Phases_start_time[] = Current_phase_start_time
                                        Phases_end_time[] = Current_phase_end_time
                                        Display ("Not able to find the entire phases, loop breaked
inside CONVERSATION loop1")
                                        Display (All the phases)
```

```
                    else
                        If(Current_phase == 'Climb' or 'Cruise')
                            Phases[] = Current_phase;
                            Durations[] = Current_Phase_duration
                            Confidences[] = Current_phase_confidence
                            Phases_start_time[] = Current_phase_start_time
                            Phases_end_time[] = Current_phase_end_time

                            Current_phase = Cruise
                            Current_phase_confidence = Confidence
                            Current_phase_start_time = Start_time
                            Current_phase_end_time = End_time
                            Current_Phase_duration = Duration;
                        else
                            Display ("Not able to find the entire phases, loop breaked
inside CONVERSATION loop2")
                            Display (All the phases)


            else
                    if(Phase != Next_phase) && (Total_Duration <
Max_phase_time_limit[current_phase])
                        Current_phase_end_time = End_time;
                        Current_Phase_duration = Total_Duration;
                    else
                        if(Phase == Next_phase) && ((Duration > 01:00) || (Confidence >
.85))
                            Phases[] = Current_phase;
                            Durations[] = Current_Phase_duration
                            Confidences[] = Current_phase_confidence
                            Phases_start_time[] = Current_phase_start_time
                            Phases_end_time[] = Current_phase_end_time

                            Current_phase = Phase
                            Current_phase_confidence = Confidence
                            Current_phase_start_time = Start_time
                            Current_phase_end_time = End_time
                            Current_Phase_duration = Duration;
                        else
                            if(Total_Duration < Max_phase_time_limit[current_phase])
                                Current_phase_end_time = End_time;
                                Current_Phase_duration = Total_Duration;


*******************************************************************************
*****************
//To find the inital phase
If(Inital_phase == NULL)
    if((Duration > 01:00) && (Confidence > .70))
                    Inital_phase = phase;
                    Current_phase = phase;
```

```
                    Current_Phase_duration = Duration;

                    Initial_phase_start_time = Start_time;
                    Initial_phase_end_time = End_time;
                    Current_phase_confidence = Confidence
                    Current_phase_start_time = Start_time
                    Current_phase_end_time = End_time

        ELSE
                IF(Inital_phase == CONVERSATION or not identified)
                    Inital_phase = NULL;
                    Total_Phase_duration = Duration;
                    Initial_phase_start_time = Start_time;
                    Initial_phase_end_time = End_time;
loop end
```

Let the extra variables be there I will do improvements later. Also there might be some blunders feel free to improve it. And also include proper commands.

If you have any doubts please free to ask. Also one important point. Phase should always follow that particular order.

Response

Got it ✅ — this is a fairly detailed algorithm that needs to be turned into a structured Python program.

Before I code it, I'd like to confirm a few **key details** to make sure the logic matches your intent:

🔍 Clarification Questions

1. **File format**

   * Each line in the `.txt` file follows this structure?

     ```
     [29:23 → 29:50] : [00:27] : Preflight    (Average Confidence: 0.948)
     ```

     * Meaning:

       * **Start time** = `29:23`
       * **End time** = `29:50`
       * **Duration** = `00:27`
       * **Phase** = `Preflight`
       * **Confidence** = `0.948` (may not exist if the phase is `CONVERSATION`)
     * Correct?

2. **Phase order constraint**
   You mentioned:

```
Preflight → TAXI → Takeoff → Climb → Cruise → Descent → Approach → Landing → TAXI → Parking
```

* Should the algorithm enforce this strict order?
  (i.e., if a “Takeoff” appears before a “TAXI”, should it be ignored or flagged?)

3. **Output format**
   What would you like the final output to look like? For example:

```python
[
  {"phase": "Preflight", "start": "29:23", "end": "30:44", "duration": "01:21", "confidence": 0.78},
  {"phase": "Climb", "start": "30:44", "end": "45:22", "duration": "14:38", "confidence": 0.49},
  ...
]
```

or just printed summaries?

4. **Conversation handling**
   The pseudocode handles short and long CONVERSATION blocks differently.

* If CONVERSATION duration `< 05:00`, it *extends the current phase*.
* If `≥ 05:00` and the current phase is *Climb* or *Cruise*, it transitions to Cruise.
* Otherwise, it breaks.
  — Is this correct as written?

5. **Time parsing**
   Times like `[29:23]` — should I assume those are **mm:ss** (not hh:mm:ss), unless they include `:` twice?

Once you confirm these, I’ll generate:
✅ A clean, well-commented Python implementation
✅ That reads from your `.txt` file, parses it, follows your algorithmic logic, and outputs total flight phases clearly.

Would you like the code to:

* **(A)** print the phases to the console,
* **(B)** write the final list of phases to a `.csv` or `.txt`,
* or **(C)** both?

Next prompt
1. Yes correct. It can also take this format
[01:06:31 → 01:07:11] : [00:40] : Takeoff (Average Confidence: 0.485)

But the only difference is that there is hour as well.

2. Yes correct, once the initial phase is found, then it should follow that order.

3. It would be good, if you can do it this way
[00:30 → 20:35] : [20:35] : Preflight (Average Confidence: 0.710)
[20:35 → 23:03] : [02:28] : Takeoff (Average Confidence: 0.644)
[23:03 → 59:37] : [36:34] : Climb (Average Confidence: 0.930)
[01:00:00 → 01:19:27] : [Duration] : Cruise
[01:19:28 → 01:33:38] : [Duration] : Descent (Average Confidence: 0.944)
[01:33:38 → 01:39:56] : [Duration] : Approach (Average Confidence: 0.688)
[01:39:56 → 01:42:05] : [Duration] : Landing (Average Confidence: 0.644)
[01:42:10 → 01:43:39] : [Duration] : TAXI (Average Confidence: 0.580)
[01:43:39 → 01:50:05] : [Duration] : Preflight (Average Confidence: 0.692)

4. yes, correct.

5. Yes correct
C. Write it into a .txt file and also show it in my console.

Response
Perfect ✅ — thank you for confirming all details!

Here's the plan before I give you the full working Python code:

🧠 Summary of What the Script Will Do

1. **Read** the `.txt` input line by line.
2. **Parse** each line into:

   * Start time, end time, duration (converted to seconds)
   * Phase name (e.g., `Preflight`, `Climb`, etc.)
   * Confidence (if available)
3. **Determine the initial phase** (per your logic).
4. **Iterate** through each subsequent record, applying your algorithmic logic:

   * Handle repeated same-phase segments.
   * Handle short/long CONVERSATION blocks.
   * Respect the **phase order** sequence.
   * Manage max/min phase time constraints.
5. **Output results**:

   * Display combined phases in the console.
   * Write them in your preferred format to an output `.txt` file.

🧩 Assumptions

* The input file is `flight_phases_input.txt`
* The output will be `flight_phases_output.txt`
* Any “CONVERSATION” entries have **no confidence** (or ignored).

✅ Here’s the full Python script

You can save this as `process_flight_phases.py` and run it directly.

```python
import re
from datetime import timedelta, datetime

# ------------------------------------------------------------
# Utility Functions
# ------------------------------------------------------------

def parse_time(t):
    """Parse time in mm:ss or hh:mm:ss format → timedelta"""
    parts = t.split(':')
    if len(parts) == 2:
        m, s = map(int, parts)
        return timedelta(minutes=m, seconds=s)
    elif len(parts) == 3:
        h, m, s = map(int, parts)
        return timedelta(hours=h, minutes=m, seconds=s)
    return timedelta(0)


def parse_line(line):
    """Parse a line from the log file"""
    pattern = (
        r"\[(.*?) → (.*?)\]\s*:\s*\[(.*?)\]\s*:\s*(\w+)"
        r"(?:\s*\(Average Confidence:\s*([0-9.]+)\))?"
    )
    match = re.search(pattern, line)
    if not match:
        return None

    start, end, duration, phase, conf = match.groups()
    start_time = parse_time(start)
    end_time = parse_time(end)
    duration_td = parse_time(duration)
    confidence = float(conf) if conf else None

    return {
        "start": start_time,
        "end": end_time,
        "duration": duration_td,
        "phase": phase.strip(),
```

```
        "confidence": confidence
    }


def format_timedelta(td):
    """Format timedelta as HH:MM:SS or MM:SS if short"""
    total_seconds = int(td.total_seconds())
    h, rem = divmod(total_seconds, 3600)
    m, s = divmod(rem, 60)
    if h > 0:
        return f"{h:02}:{m:02}:{s:02}"
    else:
        return f"{m:02}:{s:02}"


# ----------------------------------------------------------
# Main Phase Processing Logic
# ----------------------------------------------------------

def process_flight_phases(input_file, output_file):
    # Define phase order and limits
    phase_order = ["Preflight", "TAXI", "Takeoff", "Climb", "Cruise",
                   "Descent", "Approach", "Landing", "TAXI", "Parking"]

    min_limits = [
        "05:00", "01:30", "00:45", "10:00", "20:00",
        "05:00", "03:00", "00:30", "02:00", "02:00"
    ]
    max_limits = [
        "45:00", "30:50", "03:45", "40:00", "02:20:00",
        "40:00", "30:00", "03:30", "20:00", "20:00"
    ]

    min_limits = [parse_time(t) for t in min_limits]
    max_limits = [parse_time(t) for t in max_limits]

    with open(input_file, 'r') as f:
        lines = [parse_line(line) for line in f if parse_line(line)]

    if not lines:
        print("No valid data found in input file.")
        return

    phases, durations, confidences, start_times, end_times = [], [], [], [], []

    current_phase = None
    initial_phase = None
    current_confidence = 0
    total_duration = timedelta(0)
    current_start = None
    current_end = None
```

```
# ----------------------------------------------------
# Loop through parsed entries
# ----------------------------------------------------
for i, entry in enumerate(lines):
    phase = entry["phase"]
    dur = entry["duration"]
    conf = entry["confidence"]
    start = entry["start"]
    end = entry["end"]

    if not initial_phase:
        # Find initial phase
        if conf and conf > 0.70 and dur > timedelta(minutes=1):
            initial_phase = phase
            current_phase = phase
            current_confidence = conf
            current_start = start
            current_end = end
            total_duration = dur
        else:
            continue
    else:
        # Continue main logic
        total_duration += dur

        if phase == current_phase:
            # Combine repeated phase
            current_end = end
            if dur > timedelta(minutes=1) and conf:
                current_confidence = (
                    (current_confidence + conf) / 2
                    if current_confidence and conf
                    else conf or current_confidence
                )

        elif phase == "CONVERSATION":
            if dur < timedelta(minutes=5):
                # Continue current phase
                if total_duration < max_limits[phase_order.index(current_phase)]:
                    current_end = end
                else:
                    # Save current phase
                    phases.append(current_phase)
                    durations.append(total_duration)
                    confidences.append(current_confidence)
                    start_times.append(current_start)
                    end_times.append(current_end)
                    print(" ⚠️ Phase ended inside short CONVERSATION block.")
                    break
            else:
```

```
            # Long conversation handling
            if current_phase in ["Climb", "Cruise"]:
                phases.append(current_phase)
                durations.append(total_duration)
                confidences.append(current_confidence)
                start_times.append(current_start)
                end_times.append(current_end)
                current_phase = "Cruise"
                current_confidence = conf or 0
                current_start = start
                current_end = end
                total_duration = dur
            else:
                print(" ⚠️ Phase ended inside long CONVERSATION block.")
                break
        else:
            # Phase changed
            idx = phase_order.index(current_phase)
            next_expected = phase_order[idx + 1] if idx + 1 < len(phase_order) else None

            if phase == next_expected and (
                dur > timedelta(minutes=1) or (conf and conf > 0.85)
            ):
                # Save the current phase
                phases.append(current_phase)
                durations.append(total_duration)
                confidences.append(current_confidence)
                start_times.append(current_start)
                end_times.append(current_end)

                # Start new phase
                current_phase = phase
                current_confidence = conf or 0
                current_start = start
                current_end = end
                total_duration = dur
            else:
                if total_duration < max_limits[idx]:
                    current_end = end
                else:
                    phases.append(current_phase)
                    durations.append(total_duration)
                    confidences.append(current_confidence)
                    start_times.append(current_start)
                    end_times.append(current_end)

# Append last phase
if current_phase and total_duration > timedelta(0):
    phases.append(current_phase)
    durations.append(total_duration)
    confidences.append(current_confidence)
```

```
        start_times.append(current_start)
        end_times.append(current_end)

    # -------------------------------------------------------
    # Output
    # -------------------------------------------------------
    output_lines = []
    for i in range(len(phases)):
        conf_str = f"(Average Confidence: {confidences[i]:.3f})" if confidences[i] else ""
        line = (
            f"[{format_timedelta(start_times[i])} → {format_timedelta(end_times[i])}] : "
            f"[{format_timedelta(durations[i])}] : {phases[i]:<10} {conf_str}"
        )
        print(line)
        output_lines.append(line)

    with open(output_file, 'w') as f:
        f.write("\n".join(output_lines))

    print(f"\n✅ Output written to: {output_file}")

# -------------------------------------------------------
# Run script
# -------------------------------------------------------
if __name__ == "__main__":
    process_flight_phases("flight_phases_input.txt", "flight_phases_output.txt")
```

💡 Notes

* You can adjust the **confidence** and **duration** thresholds easily near the top of the loop.
* The script prints warnings if the algorithm “breaks” inside CONVERSATION logic.
* Times and durations are fully handled as real time intervals.
* Output looks exactly like your requested format.

Would you like me to modify it so that it **automatically skips invalid phase orders** (e.g., “Takeoff” before “TAXI”) — or should it **print a warning and continue** instead?

Synthetic data generation

Prompt:
Give me similar radio call
Jerovinks, 30B, request for flight level.

Response:
Excellent — that's a good example of an **en-route or climb-phase radio call**, where the pilot is **requesting a flight level (altitude clearance)** from ATC.

Let's break yours down and then build realistic variants 👇

🧭 **Your example**
```
Jerovinks, 30B, request for flight level.
```
Likely means:
> Callsign **Jerovinks 30B** (fictional) is requesting an altitude assignment or climb clearance from ATC.
> Spoken naturally: "Jerovinks three-zero-Bravo, request flight level."

✈️ **10 realistic radio-call variants (same intent: requesting flight level)**

#	Example Call	Typical Meaning
1	**Jerovinks 30B, request flight level three-five-zero.**	Asking clearance to climb to FL350.
2	**Jerovinks 30B, requesting climb to flight level two-seven-zero.**	Climb request while en route.
3	**Jerovinks 30B, requesting descent to flight level one-five-zero.**	Request to descend.
4	**Jerovinks 30B, request higher, flight level three-seven-zero if available.**	Asking for a higher cruise level.
5	**Jerovinks 30B, request lower, flight level one-nine-zero for turbulence.**	Requesting lower due to turbulence.
6	**Jerovinks 30B, ready for climb to flight level two-five-zero.**	Reporting ready to climb.
7	**Jerovinks 30B, request block flight level three-one-zero to three-three-zero.**	Asking for a block altitude range.
8	**Jerovinks 30B, request flight level three-one-zero due performance.**	Performance-related climb request.
9	**Jerovinks 30B, requesting cruise level change to flight level two-nine-zero.**	Cruise-phase level change.
10	**Jerovinks 30B, request maintain flight level two-four-zero.**	Asking to remain at a given level.

🧩 **Variants for other Lufthansa Technik client airlines (same pattern)**
```
DLH245, request flight level three-six-zero.
SWR118, request higher, flight level three-eight-zero.
AUA94, request descent to flight level one-seven-zero.
```

BEL209, ready for climb to flight level two-nine-zero.
CFG521, request lower, flight level one-nine-zero for weather.
OCN412, request block flight level three-one-zero to three-three-zero.
```
---
### 🪪 **Typical label for your dataset**

| Phrase example | Label |
| --- | --- |
| “Jerovinks 30B, request flight level three-five-zero.” | **Climb** (if requesting higher) |
| “Jerovinks 30B, request descent to flight level one-five-zero.” | **Descent** |
---
Would you like me to generate **a structured batch (e.g. 30 lines)** in your dataset format — `<sentence>  <phase>` — including a mix of climb/descent variants like these?
```